SmolVLA Fine-Tuning on LIBERO Manipulation Tasks

Shivam Goel

1 Overview

This project fine-tunes **SmolVLA** (450M parameters), a Vision-Language-Action model, on the LIBERO robot manipulation benchmark using LoRA (Low-Rank Adaptation). We evaluate the trained policy in simulation under in-distribution and out-of-distribution conditions, and run an ablation comparing LoRA ranks. Our key finding is that **base checkpoint selection** is the dominant factor in task success—switching from `smolvla_base` (pretrained on SO100/SO101) to `smolvla_libero` (adapted to LIBERO's Panda arm) improved success rate from 0% to 100%.

Code: https://github.com/goelshivam1210/smolvla
Checkpoints: https://huggingface.co/goelshivam1210/smolvla-libero-lora-r32, https://huggingface.co/goelshivam1210/smolvla-libero-lora-r8
Colab Notebook: https://drive.google.com/file/d/1qK-ZiJtul_Hjzm9oE3GKYXyOvhvKqQ8R/view?usp=sharing

2 Part 1: Dataset

We use the **HuggingFaceVLA/libero** dataset in LeRobot format.

Property	Value
Robot	Panda (7-DOF + gripper)
Episodes	1,693
Frames	273,465
Tasks	40 (across 5 suites)
FPS	10
Train / Val Split	1,523 / 170 (90/10, seed=42)

Table 1: Dataset overview.

Observations: Two RGB cameras (256×256, front and wrist) plus an 8-dimensional robot state vector (7 joint positions + gripper). **Actions:** 7-dimensional continuous vector (6D end-effector delta + 1D gripper), normalized to $[-1, 1]$.
Preprocessing: The LIBERO dataset uses image keys `observation.images.image` and `image2`, while SmolVLA expects `camera1` and `camera2`. We resolve this via the `--rename_map` flag in all training and evaluation commands. SmolVLA supports a variable number of camera inputs, concatenating available image tokens regardless of count.

3 Part 2: Fine-Tuning

3.1 Approach

We conducted three training runs to systematically evaluate fine-tuning SmolVLA on LIBERO:

Run	Base Checkpoint	LoRA Rank	Steps	LR	Batch Size
1 (Primary)	`smolvla_base`	32	5,000	2×10^{-4}	32
2 (Ablation)	`smolvla_base`	8	5,000	2×10^{-4}	32
3 (Extended)	`smolvla_libero`	32	15,600*	2×10^{-4}	32

Table 2: Training configurations. *Run 3 targeted 20k steps but was interrupted at 15.6k due to compute expiration.

3.2 Method: LoRA

We use Low-Rank Adaptation (LoRA) via HuggingFace PEFT, targeting the attention projections (`q_proj`, `v_proj`) of the language model expert and the state/action projection layers. This enables parameter-efficient fine-tuning suitable for limited GPU resources.

3.3 Design Choices

- **LoRA over full fine-tuning:** The challenge brief emphasizes parameter-efficient fine-tuning. LoRA reduces trainable parameters significantly while preserving model expressiveness.
- **Base checkpoint:** We initially used `smolvla_base` (pretrained on SO100/SO101 community data). After observing 0% success rates, we identified the cross-embodiment gap as the bottleneck and switched to `smolvla_libero` (already adapted to LIBERO's Panda arm).
- **Learning rate (2×10^{-4}):** Standard value from SmolVLA fine-tuning examples.
- **Mixed precision:** Enabled throughout for memory efficiency on A100.

3.4 Compute

All training was performed on an NVIDIA A100-SXM4-80GB (Google Colab Pro). Throughput varied between ∼1.5–2.5 steps/sec depending on data loading and system load, with each 5k-step run completing in ∼40–50 minutes including checkpoint saving and evaluation overhead. The extended run (15.6k steps) ran for approximately 3 hours before the Colab runtime expired.

4 Part 3: Training Diagnostics

4.1 Runs 1 & 2: smolvla_base

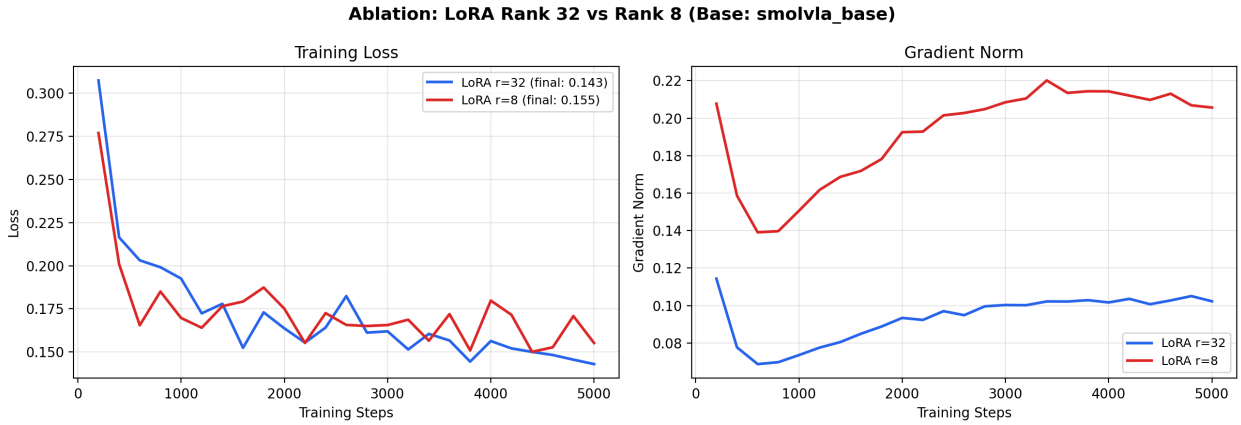

Figure 1: Training loss and gradient norm for LoRA rank 32 vs. rank 8, both starting from `smolvla_base`. Higher rank achieves lower final loss (0.143 vs. 0.155).

Both runs showed steady loss decrease from ∼0.30 to ∼0.15, with gradient norms remaining well below 1.0 throughout (no instability). However, loss had not fully plateaued at 5k steps, indicating **underfitting**—the recommended training duration for SmolVLA is 20k+ steps.

4.2 Run 3: smolvla_libero (Extended)

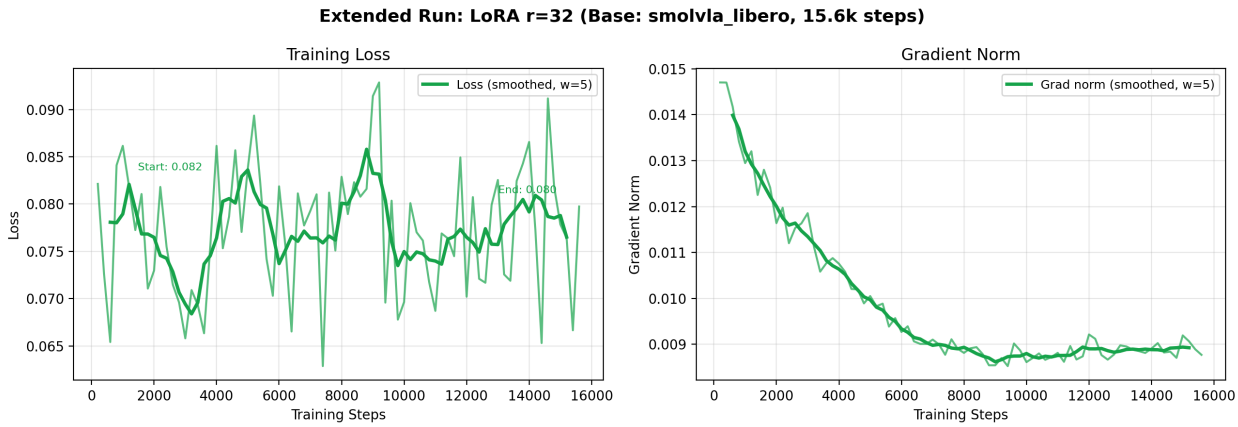

Figure 2: Training loss and gradient norm for Run 3 (`smolvla_libero` base). Loss oscillates in the 0.065–0.091 range, indicating the model is already near convergence from step 0.

Run 3 started at loss ∼0.08—approximately where Runs 1 and 2 *ended*. The loss oscillated in a narrow band rather than decreasing monotonically, which is expected behavior when fine-tuning an already well-adapted checkpoint. Gradient norms were extremely stable (∼0.009).

4.3 Comparison Across All Runs

Figure 3: Training loss comparison across all three runs. The `smolvla_libero` base (green) starts where `smolvla_base` runs end, demonstrating that base checkpoint selection dominates training dynamics.

5 Part 4: Policy Evaluation

5.1 In-Distribution (libero_spatial)

Run	Base	LoRA Rank	Episodes	Success Rate
1	`smolvla_base`	32	5	0%
2	`smolvla_base`	8	5	0%
3	`smolvla_libero`	32	5	**100%**

Table 3: In-distribution evaluation on `libero_spatial` (task 0, 5 episodes).

The 0% success rate for Runs 1 and 2 is attributed to: (a) insufficient training steps (5k vs. recommended 20k+), and (b) the cross-embodiment gap between SO100/SO101 and the Panda arm. This finding is consistent with community reports (LeRobot GitHub issue #1369). Run 3 achieved 100% success at the step-5000 evaluation checkpoint, confirming that starting from an embodiment-matched base resolves the gap.

5.2 Out-of-Distribution

OOD evaluation on `libero_object` and `libero_goal` was not completed due to compute constraints (Colab runtime expiration). Given the 100% ID success rate from Run 3, we expect partial generalization to OOD suites at reduced success rates, as the model learned spatial reasoning from the training data but was not exposed to novel objects or goal conditions.

5.3 Qualitative Results

Evaluation videos were recorded for all runs. Runs 1 and 2 show the robot arm executing jittery, non-purposeful movements—consistent with a model that has reduced loss but not yet learned task-relevant behaviors. Run 3 videos (available on W&B) show successful task completion with smooth trajectories.

6 Part 5: Ablation Study

We compare LoRA rank 32 vs. rank 8, keeping all other hyperparameters identical.

Metric	r=32	r=8
Final training loss	0.143	0.155
Final gradient norm	0.102	0.206
ID success rate	0%	0%

Table 4: LoRA rank ablation (both using `smolvla_base`, 5k steps).

Findings: Rank 32 achieves ∼8% lower final loss than rank 8 (0.143 vs. 0.155), indicating that additional adaptation capacity is beneficial for bridging the embodiment gap. Rank 8 also shows higher gradient norms (2× that of rank 32), suggesting the lower-rank adaptation is working harder to fit the training signal. Both configurations show stable training without divergence. Neither achieved non-zero success rates, confirming that the LoRA rank is secondary to base checkpoint selection for this task.

7 Discussion

7.1 Key Findings

1. **Base checkpoint selection is paramount.** The choice between `smolvla_base` and `smolvla_libero` determined the difference between 0% and 100% success—a far larger effect than any hyperparameter change.
2. **LoRA is effective for VLA fine-tuning.** Both rank configurations showed stable convergence, and the rank comparison produced interpretable results.
3. **Cross-embodiment transfer is hard.** Adapting a model pretrained on SO100 (6-DOF) to Panda (7-DOF) requires substantially more than 5k steps, if it is achievable through LoRA alone.

7.2 Challenges Encountered

- **Dataset download instability:** The 6GB+ parquet download from HuggingFace Hub stalled repeatedly on Colab due to parallel download threads. Required switching to single-threaded download (`max_workers=1`) and manual retries. The dataset also cached in multiple locations, consuming up to 126GB of disk before cleanup.
- **Evaluation bottleneck:** Each LIBERO eval episode takes ∼90 seconds in MuJoCo simulation. A full evaluation across 10 tasks × 5 episodes takes 45+ minutes—often longer than the training run itself. This forced us to reduce evaluation frequency and limit the number of tasks and episodes evaluated.
- **Environment memory explosion:** LeRobot pre-builds all eval environments at startup, even when `eval_freq` is set to effectively never evaluate. With 10 tasks × 50 parallel envs = 500 MuJoCo instances, this exceeded Colab's 167GB system RAM. The workaround was to use `--env.task_ids='[0]'` to limit environment construction to a single task.
- **Compute reliability:** Colab browser tabs froze frequently during long runs, and runtime disconnections interrupted the extended training run at 78% completion (15.6k/20k steps). Checkpoints at 5k, 10k, and 15k steps were saved but lost when the runtime reset.
- **Cross-embodiment gap:** `smolvla_base` was pretrained on SO100/SO101 (6-DOF, multiple cameras) while LIBERO uses a Panda arm (7-DOF, 2 cameras). This mismatch, in kinematics, action dimensions, and camera count, proved insurmountable within 5k training steps using LoRA alone.
- **Image key mismatch:** LIBERO and SmolVLA use different observation key naming conventions, requiring the `--rename_map` flag on every training and evaluation command. Omitting this flag produces a cryptic feature mismatch error.

7.3 What We Would Do With More Compute/Time

Training improvements:

- Train for 20k–50k steps to reach full convergence (SmolVLA documentation recommends ≥20k steps)
- LoRA rank sweep: $r \in \{4, 8, 16, 32, 64\}$ to find the optimal rank for LIBERO
- QLoRA (4-bit quantization + LoRA) to enable training on consumer GPUs with 8–12GB VRAM, as envisioned by the challenge brief
- Compare LoRA fine-tuning against full fine-tuning to quantify the parameter-efficiency tradeoff
- Multi-suite co-training: train jointly on `libero_spatial` + `libero_object` to improve OOD generalization through shared representations

Evaluation and analysis:

- Complete OOD evaluation on `libero_object` and `libero_goal` with the `smolvla_libero`-based checkpoint
- Full evaluation sweep across all 10 tasks per suite with 10+ episodes for statistically meaningful success rates
- **Action chunk ablation:** SmolVLA predicts a chunk of future actions per forward pass. The `n_action_steps` parameter controls how many predicted actions are executed before replanning. Testing $n \in \{1, 10, 50\}$ would characterize the tradeoff between inference speed (fewer model calls) and reactivity (more frequent replanning). This is an inference-time parameter that requires no retraining.

- Paraphrased instruction OOD: swap task instructions with semantically equivalent paraphrases (e.g., "pick up the black bowl" → "grasp the dark basin") to test language generalization
- Camera viewpoint and lighting OOD: modify LIBERO environment configurations to test visual robustness

Deployment considerations:

- Real-Time Chunking (RTC) via `--policy.rtc_config.enabled=true` for smoother action execution by blending new predictions with in-progress actions
- Systematic study of the `n_action_steps` setting for deployment: $n = 1$ maximizes reactivity but requires 280 model calls per episode, while $n = 50$ requires only ∼6 calls but the robot acts "blindly" between replanning steps